



COPower Technical Documentation

Centria University of Applied Sciences

Contents

1	Server	4
1.1	Installing Nginx	4
1.2	Installing Let's Encrypt HTTPS certificate.....	4
1.3	Installing Nginx site configuration file	5
1.4	Process manager	6
1.4.1	Installing NVM (Node Version Manager) and Node.js	6
1.4.2	Installing PM2	6
2	Dashboard	6
2.1	Client	7
2.1.1	Installing	7
2.1.2	Building	7
2.2	Server	7
2.2.1	Installing	7
2.2.2	Building	7
3	Services	9
3.1	Biogas Plant	9
3.2	Fludia electricity meter	9
3.3	Electricity spot price	9
3.4	OPC UA Client	9
3.4.1	Generating certificate	9
3.4.2	Setting up OPC UA server on the PLC	10
3.5	Prediction	11
3.6	Current and forecast weather	11
4	OpenVPN	11
4.1	Setting Up OpenVPN Server	11
4.1.1	Install OpenVPN and Easy-RSA	11
4.1.2	Create a Public Key Infrastructure (PKI) folder	11
4.1.3	Prepare Easy-RSA	11
4.1.4	Set certificate settings	11
4.1.5	Initialise PKI	12
4.1.6	Build the CA	12
4.1.7	Create server certificate and key	12
4.1.8	Generate Diffie-Hellman parameters for the certificate	12

4.1.9	Generate HMAC key for TLS authentication.....	13
4.1.10	Copy the generated certificate files to the OpenVPN directory.....	13
4.1.11	Create the OpenVPN server configuration.....	13
4.1.12	Create Client-Specific Configuration Directory (CCD) files.....	14
4.1.13	Enable IP forwarding.....	14
4.1.14	Enable and start OpenVPN server	14
4.1.15	Configure firewall	15
4.1.15.1	Microsoft Azure	15
4.1.15.2	UFW (Uncomplicated Firewall).....	15
4.1.15.3	nftables	16
4.2	Setting Up OpenVPN client.....	16
4.2.1	Create OpenVPN client certificate.....	16
4.2.2	Create OpenVPN client configuration.....	17
4.2.3	Setting up OpenVPN Client using ASUS 4G-AX56 router.....	18
4.2.3.1	Create OpenVPN client profile.....	18
5	Network Time Protocol (NTP).....	19
5.1	Settings up NTP server	19
5.1.1	Installing NPT server service	19
5.1.2	Configuring NTP server service	19
5.1.3	Configuring server firewall.....	20
5.1.3.1	Microsoft Azure	20
5.1.3.2	UFW (Uncomplicated Firewall).....	20
5.1.3.3	nftables	21
5.1.4	Broadcasting NTP feature to clients	21
5.2	Setting up NTP client.....	22
5.2.1	Siemens 1215c PLC.....	22
5.2.2	KTP400 Basic PN HMI panel.....	22

1 Server

CoPower is using a Microsoft Azure provided Virtual Machine B2als_V2 (2 vCPU, 4 GB ram) operating on Ubuntu 22.04 LTS.

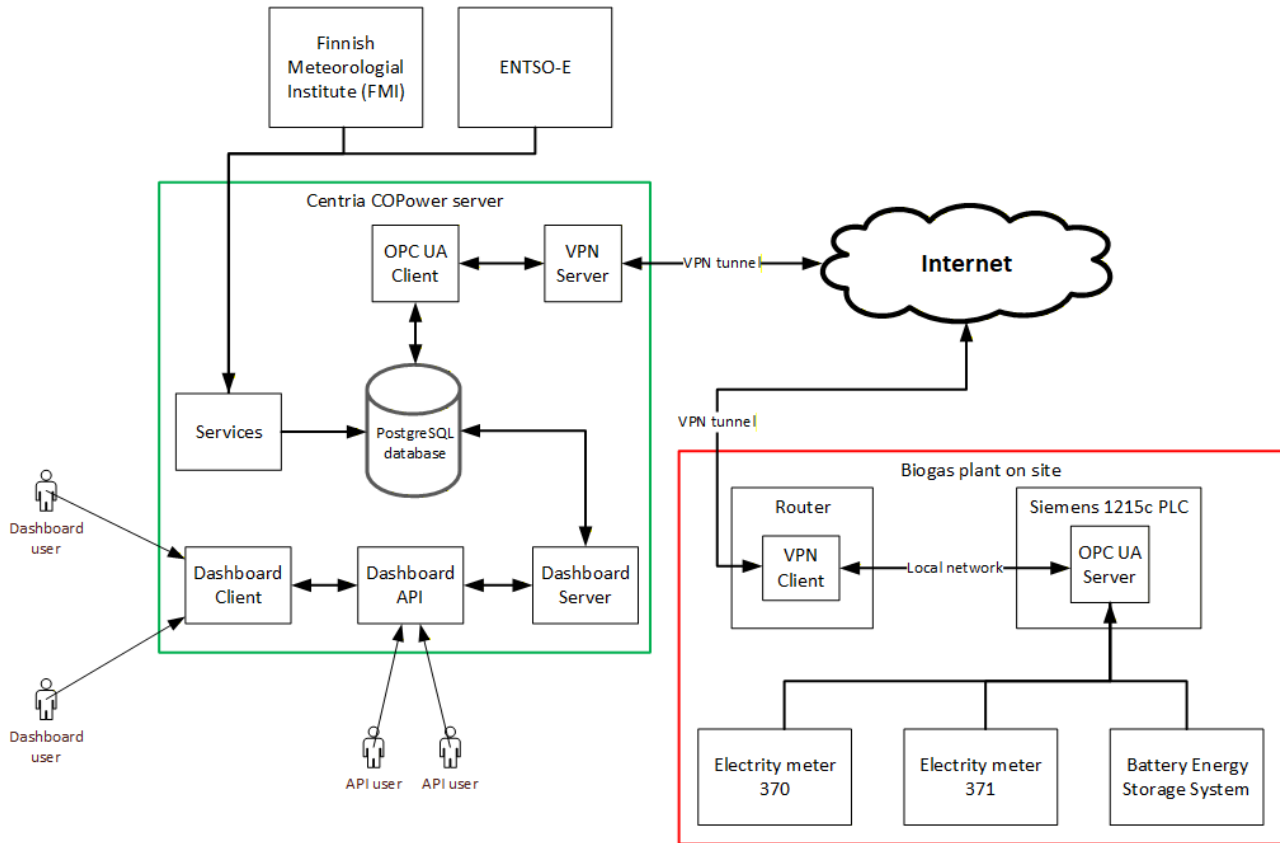


Figure 1. Diagram of the architecture

1.1 Installing Nginx

Server is using Nginx as HTTP web server for use with Dashboard client and server. You can install it with the following command.

```
sudo apt install nginx
```

1.2 Installing Let's Encrypt HTTPS certificate

Server is using Let's Encrypt which uses Certbot to automatically manage the installed certificate as the provider for the HTTPS certificate which can be installed with the following steps.

1. Install Certbot which manages the Let's Encrypt certificate
`sudo snap install --classic certbot`
2. Activate Certbot on the server
`sudo certbot --nginx`

3. Test automatic certificate renew
sudo certbot renew --dry-run

Note: do not close port 80 (HTTP) after that the certbot would be blocked from renewing the certificate and it will then expire.

1.3 Installing Nginx site configuration file

Create a *https.conf* site configuration file to the */etc/nginx/sites-available* with the following content

```
server {
    listen 443 ssl;
    server_name SERVER_NAME;

    ssl_certificate /etc/letsencrypt/live/SERVER_NAME/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/SERVER_NAME/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location /dashboard/app/ {
        root /var/www/vue-app;
        index index.html;
        try_files $uri $uri/ /dashboard/app/index.html;
    }

    location /dashboard {
        add_header 'Access-Control-Allow-Origin' '*';
        add_header 'Access-Control-Allow-Credentials' 'true';
        add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS';
        add_header 'Access-Control-Allow-Headers' 'Authorization,Keep-Alive,User-Agent,X-
Requested-With,If-Modified-Since,Cache-Control,Content>

        if ($request_method = 'OPTIONS') {
            return 204;
        }

        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot|otf|json)$ {
        root /var/www/vue-app;
        expires max;
    }
}
```

```
    log_not_found off;  
}  
  
error_page 404 /dashboard/app/index.html;  
}
```

Activate the created configuration file and restart nginx for it to take effect with the following commands.

```
sudo ln -s /etc/nginx/sites-available/https.conf /etc/nginx/sites-enabled/https.conf  
sudo systemctl reload nginx
```

1.4 Process manager

Server is running a PM2 process manager for Dashboard server and OPC UA client.

1.4.1 Installing NVM (Node Version Manager) and Node.js

NVM (Node Version Manager) is used to manage the Node.js version, it can also alternatively be installed directly, and it can be installed with the following commands.

1. Download and install nvm, optionally check what the newest NVM version is
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
2. Restart the command line shell
\. "\$HOME/.nvm/nvm.sh"
3. Download and install the latest LTS (Long-Term Support)
nvm install --lts
4. If the node is not activated afterwards
nvm use --lts

1.4.2 Installing PM2

PM2 is a process manager that can automatically restart a process if it fails or crashes at any point and it can be installed with the following commands.

1. Install PM2
npm install pm2 -g
2. Install log rotation module for PM2 which by default keeps all the logs without rotating to add log rotation
pm2 install pm2-logrotate

Any part of the server using PM2 describes how to start the process with it.

2 Dashboard

This section contains information about the client and server side of the Dashboard view.

2.1 Client

Client of the Dashboard is built using Vue3 and the production build is running on the server under the “vue-app” folder inside the “www” folder of Nginx and it can be accessed with *https://SERVER_URL/dashboard/app*.

2.1.1 Installing

On the source code root folder first run “npm i” to install all the required npm packages for the dashboard. If npm is not missing install Node.js from the official site, during the project version 24.13.0 was used.

2.1.2 Building

First set the environment variables according to example files if not set.

Building the Dashboard client is done by running “npm run build” command on the root of the source code folder. After this the “dist” folder includes the fully built version of the Dashboard client that can be installed on the server.

2.2 Server

Server of the Dashboard is built using .Net 10 with Microsoft Visual Studio 2026 and the production build of the application is running on the server on the “<USER_HOME_ROOT/Dashboard/API” with .NET 10 runtime through Nginx.

2.2.1 Installing

To run the production version of the server application .Net 10 runtime is needed and it can be installed with the following commands.

1. .NET 10 is available on the Ubuntu .NET Backports repository, first add the repository
sudo add-apt-repository ppa:dotnet/backports
2. Install the .NET 10 runtime for running the published .NET 10 application
sudo apt-get update && sudo apt-get install -y aspnetcore-runtime-10.0

2.2.2 Building

Visual Studio 2026 is used to build the server application.

Start by copying or renaming “env_example.json” to “env.json” and fill them with the proper values. The following section describes what the different environment variables are.

- AppSettings
 - o BasePath
 - What is the base path for the API
 - o ClientAPIKey
 - API key that is used by the client to contact the API server
 - o LoggingActive
 - Activate extra logging if needed
 - o ResultsPerPage
 - How many results to return per page for the client
 - ConnectionStrings
 - o CopowerDB
 - PostgreSQL connection string which includes postgresql server address, port, database name, user id and password
 - o CommonDB
 - PostgreSQL connection string which includes postgresql server address, port, database name, user id and password
 - o CommondataDB
 - PostgreSQL connection string which includes postgresql server address, port, database name, user id and password
 - Email
 - o fromEmail
 - Email address of the sender
 - o fromName
 - Name of the sender
 - o clientId
 - Azure Cliend Id of the sender from Azure
 - o clientSecret
 - Secret of the sender from Azure
 - o tenantId
 - Tenant Id of the sender from Azure
- },
- Frontend
 - o BaseUrl
 - Base url for the Dashboard client for example what base url to direct when using forgotten password to password renew page of the client
 - Jwt
 - o Audience
 - This Id identifies the recipients that the JWT is intended for this Dashboard server
 - o Issuer
 - This Id allows the Dashboard client to verify the token’s origin and trust that it has been issued by the Dashboard server
 - o Secret
 - Random, high-entropy string used by Dashboard server to sign and verify tokens, ensuring data integrity

3 Services

Server has multiple different independently running services performing different functions of the server like data fetching or updating.

Services have been created with Python 3.12 and each of the folders have a requirements.txt file which packages are required to run the service.

3.1 Biogas Plant

This service is a Python 3 script which retrieves the biogas plant measurements from the manufacturer's cloud and saves it into local PostgreSQL database and uses two supporting JSON file structures to determine what measurements to get, where to save it and how are they handled or post processed.

3.2 Fludia electricity meter

This service connects to the manufacturer of the Fludia electricity meter and fetches the measurements saving it into a local PostgreSQL database.

This part is not used and has been replaced by the energy meters connected to PLC.

3.3 Electricity spot price

This service fetches the electricity spot prices daily which is converted from €/MWh to cnt/kWh

3.4 OPC UA Client

This includes information on how to generate a certificate for OPC UA client to connect to a OPC UA server on a Siemens 1215C PLC.

3.4.1 Generating certificate

To generate a certificate for OPC UA client connecting to OPC UA server, first create a certificate configuration file and name it "cert-config.cnf" and set the following content while changing values as needed.

```
[ req ]
default_bits = 2048
default_md = sha256
distinguished_name = subject
req_extensions = req_ext
```

```
x509_extensions = req_ext
string_mask = utf8only
prompt = no
```

```
[ req_ext ]
basicConstraints = CA:FALSE
nsCertType = client, server
keyUsage = nonRepudiation, digitalSignature, keyEncipherment, dataEncipherment, keyCert-
Sign
extendedKeyUsage = serverAuth, clientAuth
nsComment = "OpenSSL Generated Certificate"
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid,issuer
subjectAltName = URI:urn:opcua:python,IP:192.168.0.1
```

```
[ subject ]
countryName = COUNTRY
stateOrProvinceName = PROVINCE_NAME
localityName = LOCALITY_NAME
organizationName = ORGANISATION_NAME
commonName = COMMON_NAME
```

```
[ alt_names ]
URI.1 = urn:opcua:python
IP.1 = 192.168.0.1
```

Using OpenSSL generate private and public keys with the following command.

```
openssl req -x509 -newkey rsa:2048 -sha256 -keyout copower_opcua_private_key.pem -out
copower_opcua_cert.pem -days 3650 -nodes -config cert-config.cnf
```

In TIA Portal project open "Certificate manager" from "Security features" under "Security settings" and open the "Trusted certificates and root certificates" subpage. Then import the generated public and private keys (in this order) and add it to the PLC's OPC UA's "Trusted clients" list after which the PLC will trust the certificate.

3.4.2 Setting up OPC UA server on the PLC

Under PLC OPC UA security settings disable "Enable guest authentication" which is needed for the OPC UA server to work and create a new user with password for "User management". This user will be used when connecting to the OPC UA server with a client.

Finally open "OPC UA communication" menu and add a new server interface. Open the newly created interface and find the tags you want from the "OPC UA elements" view and drag them to the "OPC UA server interface" view, after that they will be available on the OPC UA for connecting clients.

3.5 Prediction

3.6 Current and forecast weather

4 OpenVPN

4.1 Setting Up OpenVPN Server

This will setup OpenVPN for the server and clients connecting to it.



Figure 2. Diagram of the OpenVPN network

4.1.1 Install OpenVPN and Easy-RSA

Run the following command line commands to install OpenVPN and Easy-RSA.

```
sudo apt update
sudo apt install openvpn easy-rsa -y
```

4.1.2 Create a Public Key Infrastructure (PKI) folder

Create a folder for the OpenVPN Certificate Authority (CA) folder and navigate to the folder

```
make-cadir ~/openvpn-ca
cd ~/openvpn-ca
```

4.1.3 Prepare Easy-RSA

Prepare the installed Easy-RSA by copying it to the OpenVPN CA folder for easier access

```
mkdir easy-rsa
cp -r /usr/share/easy-rsa/* ./easy-rsa
```

4.1.4 Set certificate settings

Set the settings for the OpenVPN certifications by creating a file.

```
nano ./easy-rsa/vars
```

Add the following lines to the file to set the settings and update your organization contact data, this will set certificate expiration to 6 months and Certificate Revocation List (CRL) expires every 30 days.

```
set_var EASYRSA_REQ_COUNTRY "COUNTRY"  
set_var EASYRSA_REQ_PROVINCE "PROVINCE"  
set_var EASYRSA_REQ_CITY "CITY"  
set_var EASYRSA_REQ_ORG "ORGANISATION"  
set_var EASYRSA_REQ_EMAIL "EMAIL"  
set_var EASYRSA_REQ_OU "ORGANISATION_UNIT"  
set_var EASYRSA_CERT_EXPIRE 1825  
set_var EASYRSA_CRL_DAYS 30  
set_var EASYRSA_KEY_SIZE 3072
```

4.1.5 Initialise PKI

Initialise PKI with the following command.

```
./easy-rsa/easyrsa init-pki
```

4.1.6 Build the CA

Build the CA certificate with the following command.

```
./easy-rsa/easyrsa build-ca
```

4.1.7 Create server certificate and key

Generate a server certificate and key with the following commands, set the common name of the certificate to identify the OpenVPN server for example "MyOrganisationServer"

```
./easy-rsa/easyrsa gen-req server nopass  
./easy-rsa/easyrsa sign-req server server
```

4.1.8 Generate Diffie-Hellman parameters for the certificate

Generate Diffie-Hellman parameters for ephemeral key exchange in TLS with the following command.

```
./easy-rsa/easyrsa gen-dh
```

4.1.9 Generate HMAC key for TLS authentication

Generate HMAC key for OpenVPN shared secret key used for TLS authentication with the following command.

```
openvpn --genkey secret ta.key
```

4.1.10 Copy the generated certificate files to the OpenVPN directory

Copy the generated certificates to the OpenVPN directory with the following command.

```
sudo cp pki/ca.crt pki/private/server.key pki/issued/server.crt pki/dh.pem ta.key /etc/openvpn/
```

4.1.11 Create the OpenVPN server configuration

Create the configuration file used with OpenVPN server.

```
sudo nano /etc/openvpn/server.conf
```

Use the following content for the configuration file.

```
port 1194
proto udp
dev tun

ca ca.crt
cert server.crt
key server.key
dh dh.pem
tls-auth ta.key 0
auth SHA256
cipher AES-256-GCM

ifconfig-pool-persist ipp.txt

topology subnet
server 10.8.0.0 255.255.255.0

# Allow VPN clients to reach each other
client-to-client

# Add a kernel route to the ASUS LAN via the VPN
route 192.168.0.0 255.255.255.0
```

```
# >>> REQUIRED so CCD files (and your iroute) are honored
client-config-dir /etc/openvpn/ccd
```

```
#push "redirect-gateway def1 bypass-dhcp"
push "dhcp-option DNS 1.1.1.1"
```

```
keepalive 10 120
persist-key
persist-tun
```

```
user nobody
group nogroup
```

```
# Logging
log-append /var/log/openvpn.log
status /var/log/openvpn-status.log
verb 3
```

4.1.12 Create Client-Specific Configuration Directory (CCD) files

Add the CCD file for each connected client to setup client specific settings the name of the file should match the common name of the client on the certificate that will be used for them. This will tell OpenVPN server that there are clients behind the OpenVPN client which can be accessed.

```
sudo nano /etc/openvpn/ccd/MyOpenVPNClient1
```

Add the following lines to the file.

```
# Tell OpenVPN that 192.168.0.0/24 is behind this client
iroute 192.168.0.0 255.255.255.0
```

4.1.13 Enable IP forwarding

Allow different OpenVPN clients to reach each other by adding the following line to the /etc/sysctl.conf file.

```
net.ipv4.ip_forward = 1
```

4.1.14 Enable and start OpenVPN server

Enable and start the OpenVPN server with the following commands.

```
sudo systemctl enable openvpn@server
sudo systemctl start openvpn@server
sudo systemctl status openvpn@server
```

4.1.15 Configure firewall

Configure the firewall on the server to allow for valid OpenVPN clients to connect to the server using OpenVPN.

4.1.15.1 Microsoft Azure

If using Microsoft Azure virtual machine (VM) create a firewall rule with the following steps.

1. Navigate to the <https://portal.azure.com/>
2. Find your virtual machine from the list of virtual machines and open it by clicking on the name
3. From the left side navigation menu open “Networking” and select “Network settings” menu.
4. Click “Create port rule” to create a new firewall rule for inbound traffic.
5. Set the following settings
 1. Source = IP Addresses
 2. Source IP addresses/CIDR ranges = the IP address of your OpenVPN client(s) separated by comma if multiple for example 100.101.102.1,101.101.102.1
 3. Source port ranges = * (star)
 4. Destination = Any
 5. Service = Custom
 6. Destination port ranges = 1194
 7. Protocol = UDP
 8. Action = Allow
 9. Priority = Any number which is not used, can be set to default value
 10. Name = Name of the rule for example “Allow-OpenVPN-UDP-1194”
 11. Description = Description of the rule in addition to the name
6. Click “Save” to save the new firewall rule and it will be activated automatically
7. (Optional) Edit the firewall rule if something has changed by clicking on the name of the rule you provided

4.1.15.2 UFW (Uncomplicated Firewall)

If using UFW as firewall which is used by example Ubuntu set the rule in the following command.

```
sudo ufw allow 1194/udp
```

Allow traffic and NAT for the routed VPNs.

```
sudo nano /etc/ufw/before.rules
```

Add the following lines before the ***filter** line

```
*nat
```

```
:POSTROUTING ACCEPT [0:0]
```

```
# NAT OpenVPN traffic  
-A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
```

```
COMMIT
```

Reload the UFW with the following command.

```
sudo ufw reload
```

Verify that the firewall rule has been added and used successfully, and you should see it on the list of allowed connections.

```
sudo ufw status numbered
```

4.1.15.3 nftables

If using nftables run the following commands to allow inbound OpenVPN traffic.

```
sudo nft add rule inet filter input udp dport 1194 accept
```

Persist the rule through server reboot by adding it to the nftables configuration.

```
sudo nano /etc/nftables.conf
```

Add the following line to the configuration

```
udp dport 1194 accept
```

Restart nftables for the rule to take effect.

```
sudo systemctl reload nftables
```

4.2 Setting Up OpenVPN client

Here we create and set the configurations and settings for the OpenVPN clients.

4.2.1 Create OpenVPN client certificate

Generate a client certificate and key for each of the clients used connecting to server and copy them to the client config folder with the following commands, set the common name of the certificate to identify the OpenVPN client(s) for example "MyOpenVPNClient1"

```
cd ~/openvpn-ca  
./easyrsa gen-req client1 nopass
```



```
./easysrsa sign-req client client1
cp pki/private/client1.key pki/issued/client1.crt pki/ca.crt ta.key ~/client-configs/
```

4.2.2 Create OpenVPN client configuration

Create OpenVPN ovpn configuration file for example with “client1.ovpn” as name with the following content and install it on the client device using OpenVPN and repeat for each used client. Replace placeholder server address with the real IP address and contents of the generated certificates in the appropriate place in bolded text.

```
client
dev tun
proto udp
remote SERVER_IP 1194

resolv-retry infinite
nobind

persist-key
persist-tun

ca ca.crt
cert client1.crt
key client1.key
tls-auth ta.key 1

cipher AES-256-GCM
auth SHA256
remote-cert-tls server

# Needed when using TLS-auth key
key-direction 1

# --- CA CERTIFICATE ---
<ca>
-----BEGIN CERTIFICATE-----
CONTENT OF CA.CRT
-----END CERTIFICATE-----
</ca>

# --- CLIENT CERTIFICATE ---
<cert>
-----BEGIN CERTIFICATE-----
BOTTOM PART OF THE CLIENT1.CRT CONTENT
-----END CERTIFICATE-----
</cert>

# --- CLIENT PRIVATE KEY ---
<key>
```

```
-----BEGIN PRIVATE KEY-----  
CONTENT OF CLIENT1.KEY  
-----END PRIVATE KEY-----  
</key>  
  
# --- TLS-AUTH KEY ---  
<tls-auth>  
#  
# 2048 bit OpenVPN static key  
#  
-----BEGIN OpenVPN Static key V1-----  
CONTENT OF TA.KEY  
-----END OpenVPN Static key V1-----  
</tls-auth>
```

verb 3

4.2.3 Setting up OpenVPN Client using ASUS 4G-AX56 router

CoPower is using ASUS 4G-AX56 router as OpenVPN client the following instructions are valid only for this model, other ASUS router models might be similar and for other routers follow the instructions of the manufacturer how to setup OpenVPN client (first make sure OpenVPN is supported).

4.2.3.1 Create OpenVPN client profile

Using OpenVPN requires that you create an OpenVPN profile which will then be used for connecting the server.

1. Open the administrator view of the router
2. Navigate to “VPN” page on the “Advanced settings” section on the left side
3. Navigate to “VPN Client” subpage
4. Click “Add profile” button
5. Select “OpenVPN” as the used VPN
6. Set the name of the profile for example “MyOrganisationVPN”
7. Upload .opvn (for example “client1.opvn”) created on section 2.2
8. Click “OK” to create the VPN client profile
9. Activate the created VPN profile
10. If the connection fails or is not connecting you can check the error that might show up from the “General Log” subpage from “System Log” page on the advanced settings

After VPN connection has been successfully established the routing table on the router should change to indicate a VPN connection which can be seen from the “Routing Table” subpage from “System Log” on the advanced settings. The routing table should resemble to the following, some IP address could be different based on what IP address the VPN is using and what IP address range the router’s DHCP is using.

Destination	Gateway	Genmask	Iface	Explanation (not shown on router)
default	80.0.0.1	0.0.0.0	usb0	Default internet connection
10.8.0.0	*	255.255.255.0	tun15	OpenVPN server
80.0.0.0	*	255.0.0.0	usb0	
80.0.0.1	*	255.255.255.255	usb0	
192.168.0.0	*	255.255.255.0	br0	DHCP clients behind this router
195.74.0.47	80.0.0.1	255.255.255.255	usb0	
195.197.44.100	80.0.0.1	255.255.255.255	usb0	

5 Network Time Protocol (NTP)

NTP is used to keep track of the accurate current date and time by all devices (Linux, Windows, Mac, Android, iOS, ...). Without it the device more or less have to “guess” how much time has passed by their own real-time clock (RTC) which is not accurate enough. Without connection to a very accurate NTP server the devices clock will very slowly start to drift and for example after a month the clock will be off depending on the accuracy of the RTC by several minutes if not more.

Since the PLC and the HMI screen don't have connection to the internet or to a global NTP server to ask what the current date and time is they are unable to keep track of time accurately. As a solution the CoPower project server, which itself is a NTP client to a global NTP server, has been set as a NTP server to the PLC and HMI who can then connect to it and update the date and time.

5.1 Settings up NTP server

This setup the NTP functionality for the server to act as NTP server for clients.

5.1.1 Installing NPT server service

Install the chrony linux package providing NTP server functionality for the CoPower server with the following command.

```
sudo apt install chrony
```

5.1.2 Configuring NTP server service

Configuring the NTP service allows it to be limited to being used in only the wanted IP addresses used on the OpenVPN network.

First open the configuration file of chrony.

```
sudo nano /etc/chrony/chrony.conf
```

Then add the following lines into the configuration file, replace the ip address if used ones are different.

```
allow 192.168.0.0/24
allow 10.8.0.0/24
bindaddress 10.8.0.1
bindcmdaddress 127.0.0.1
```

5.1.3 Configuring server firewall

Configure the firewall on the server to allow for only the valid OpenVPN clients to connect to the NTP server on the server.

5.1.3.1 Microsoft Azure

If using Microsoft Azure virtual machine (VM) create a firewall rule with the following steps.

8. Navigate to the <https://portal.azure.com/>.
9. Find your virtual machine from the list of virtual machines and open in by clicking on the name.
10. From the left side navigation menu open “Networking” and select “Network settings” menu.
11. Click “Create port rule” to create a new firewall rule for inbound traffic.
12. Set the following settings
 1. Source = IP Addresses
 2. Source IP addresses/CIDR ranges = the ip address of your OpenVPN client(s) separated by comma if multiple for example 100.101.102.1,101.101.102.1
 3. Source port ranges = * (star)
 4. Destination = Any
 5. Service = Custom
 6. Destination port ranges = 123
 7. Protocol = UDP
 8. Action = Allow
 9. Priority = Any number which is not used, can be set to default value
 10. Name = Name of the rule for example “Allow-NTP-UDP-1194”
 11. Description = Description of the rule in addition to the name
13. Click “Save” to save the new firewall rule and it will be activated automatically.
14. (Optional) Edit the firewall rule if something has changed by clicking on the name of the rule you provided.

5.1.3.2 UFW (Uncomplicated Firewall)

If using UFW as firewall which is used by example Ubuntu set the rule in the following command.

```
sudo ufw allow 123/udp
```

Allow traffic and NAT for the routed VPNs.

```
sudo nano /etc/ufw/before.rules
```

Add the following lines before the ***filter** line, replace the ip address if used ones are different.

```
*nat
:POSTROUTING ACCEPT [0:0]

# NTP traffic
-A POSTROUTING -s 192.168.0.0/24 -o eth0 -j MASQUERADE
-A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE

COMMIT
```

Reload the UFW with the following command.

```
sudo ufw reload
```

Verify that the firewall rule has been added and used successfully, and you should see it on the list of allowed connections.

```
sudo ufw status numbered
```

5.1.3.3 nftables

If using nftables run the following commands to allow inbound OpenVPN traffic.

```
sudo nft add rule inet filter input udp dport 123 accept
```

Persist the rule through server reboot by adding it to the nftables configuration.

```
sudo nano /etc/nftables.conf
```

Add the following line to the configuration

```
udp dport 123 accept
```

Restart nftables for the rule to take effect.

```
sudo systemctl reload nftables
```

5.1.4 Broadcasting NTP feature to clients

To broadcast of the available NTP feature to the OpenVPN clients open the OpenVPN configuration file found at `/etc/openvpn/server.conf` and add the following lines into it.

```
# Act as NTP server  
push "dhcp-option NTP 10.8.0.1"
```

5.2 Setting up NTP client

This setup the NTP functionality for the client to connect to the NTP server.

5.2.1 Siemens 1215c PLC

The following steps sets up NTP client functionality for Siemens 1215c PLC.

1. Open "TIA Portal" and open the project
2. From the sidebar open "PLC_1 [CPU 1215C DC/DC/DC]" (or if the name has been changed to something else) folder.
3. Open "Device configuration".
4. Double click on the PLC cpu on the visualisation
5. From the bottom configuration menu open "PROFINET interface [X1]".
6. Open "Time-of-day suncronization" settings.
7. Click and activate "Enable time synchronization via NTP server" checkbox
8. Set "Server 1" as 10.8.0.1 (or what the server's OpenVPN address is).
9. Set "Update interval" as it's needed.
10. Load the new configuration to the PLC.
11. After this the PLC should be able to connect to the NTP server through the OpenVPN tunnel.

5.2.2 KTP400 Basic PN HMI panel

The following steps sets up NTP functionality for KTP400 Basic PN HMI panel.

1. On the configuration screen on the HMI click "Settings".
2. Click and open "Date & Time" category
3. Activate NTP client by switching "Use NTP" setting to ON position.
4. Set "Update rate" as it's needed.
5. Click "Add server".
6. Set the server as 10.8.0.1 (or what the server's OpenVPN address is) and click enter.
7. After this the NTP server connection status should switch from red to green to indicate successful connection.